EXPERIMENT 9

Objective: To develop a responsive web-based input form that utilizes client-side logic to verify user data. By integrating **Validator.js**, the application identifies improperly formatted email addresses and phone numbers, displaying descriptive error messages to improve the overall user experience.

Implementation Procedure

Step 1: Constructing the HTML Framework

In this step, we define the structure of the form. Each input field is paired with a specific `<span>` element designed to hold error messages. We also include the CDN link for the **Validator.js** library to enable advanced validation functions without writing complex regular expressions manually.

`index.html`

HTML

```
None
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width,
initial-scale=1.0">
    <title>Form Validation Project</title>
    <link rel="stylesheet" href="style.css">
</head>
<body>
    <div class="container">
        <form id="myForm">
            <h2>Contact Information</h2>
```

```
            <label for="email">Email Address:</label>
            <input type="email" id="email" name="email"
required placeholder="example@mail.com">
            <span id="emailError" class="error"></span>

            <label for="phone">Phone Number:</label>
            <input type="text" id="phone" name="phone"
required placeholder="Enter mobile number">
            <span id="phoneError" class="error"></span>

            <button type="submit">Submit Data</button>
        </form>
    </div>

    <script
src="https://cdnjs.cloudflare.com/ajax/libs/validator/13.6.0/v
alidator.min.js"></script>
    <script src="script.js"></script>
</body>
</html>
```

Step 2: Enhancing the Interface with CSS

The styling ensures the form is centered, professional, and readable. Crucially, we define the `.error` class with a distinct red color to ensure that validation warnings are immediately visible to the user.

`style.css`

CSS

```
None
body {
```

```
    font-family: 'Segoe UI', Tahoma, Geneva, Verdana,
sans-serif;
    background-color: #f4f4f4;
    display: flex;
    justify-content: center;
    align-items: center;
    height: 100vh;
    margin: 0;
}

.container {
    background-color: white;
    padding: 30px;
    border-radius: 8px;
    box-shadow: 0 4px 10px rgba(0, 0, 0, 0.1);
    width: 100%;
    max-width: 400px;
}

h2 {
    margin-top: 0;
    font-size: 1.5rem;
    color: #333;
}

form {
    display: flex;
    flex-direction: column;
}

label {
    margin-bottom: 5px;
    font-weight: bold;
```

```
}

input {
    margin-bottom: 5px;
    padding: 12px;
    border: 1px solid #ccc;
    border-radius: 4px;
    font-size: 1rem;
}

button {
    margin-top: 10px;
    padding: 12px;
    background-color: #28a745;
    color: white;
    border: none;
    border-radius: 4px;
    font-size: 1rem;
    cursor: pointer;
    transition: background 0.3s ease;
}

button:hover {
    background-color: #218838;
}

.error {
    color: #d9534f;
    font-size: 0.85em;
    margin-bottom: 10px;
    height: 1.2em; /* Prevents layout jump when text appears
*/
}
```

Step 3: Implementing Validation Logic with JavaScript

The script intercepts the form's submission event. It extracts the input values, clears any existing errors, and performs a two-stage check:

1. **Email Check:** Uses `validator.isEmail()` to confirm the string follows standard email conventions.
2. **Phone Check:** Uses `validator.isMobilePhone()` to verify the number against global mobile formats.

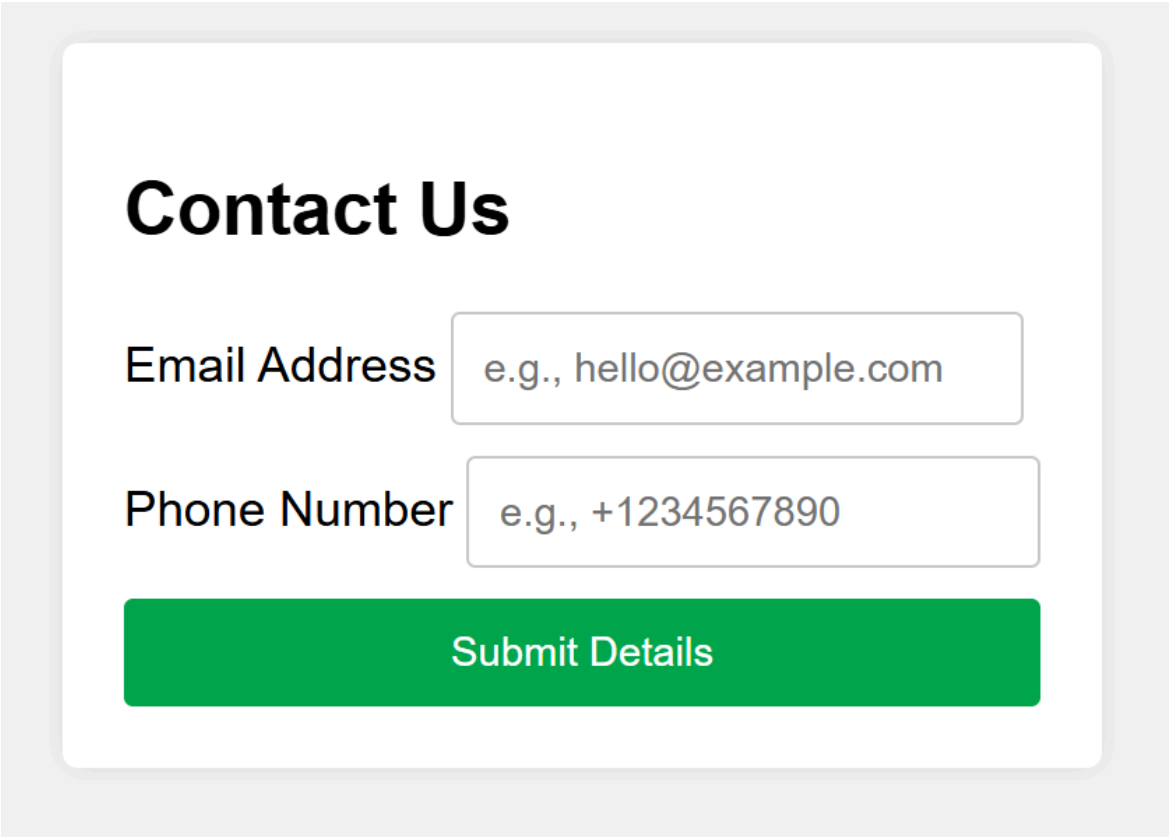
If all checks pass, the data is logged to the console; otherwise, specific error messages are injected into the UI.

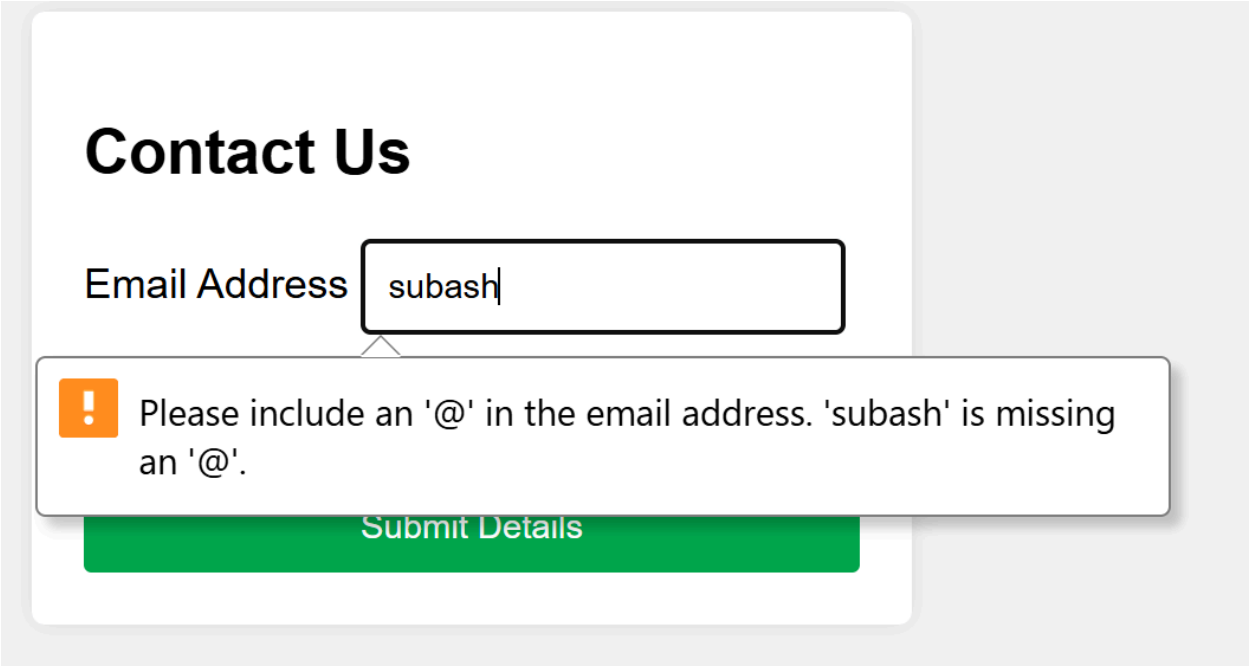
`script.js`

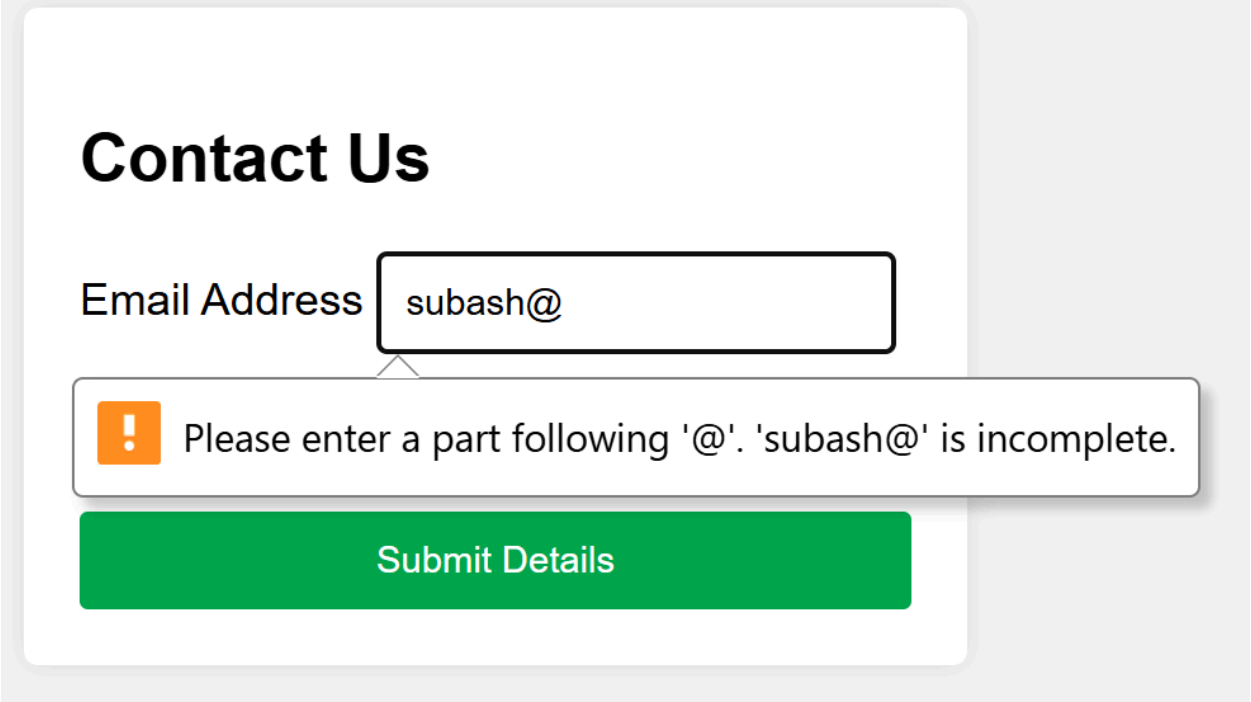
JavaScript

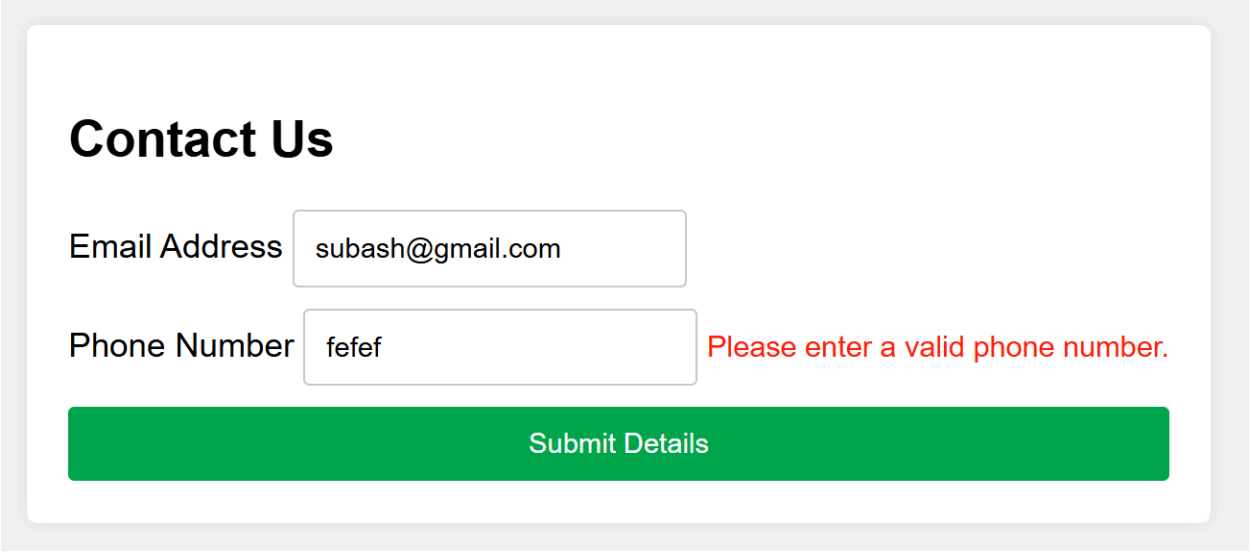
None
```
document.getElementById('myForm').addEventListener('submit',
function (e) {
    // Prevent the default form submission (page reload)
    e.preventDefault();

    // Retrieve input values and error display elements
    const email = document.getElementById('email').value;
    const phone = document.getElementById('phone').value;
    const emailError = document.getElementById('emailError');
    const phoneError = document.getElementById('phoneError');

    // Reset previous error states
    emailError.textContent = '';
    phoneError.textContent = '';

    let isFormValid = true;

    // Execute Email Validation
```

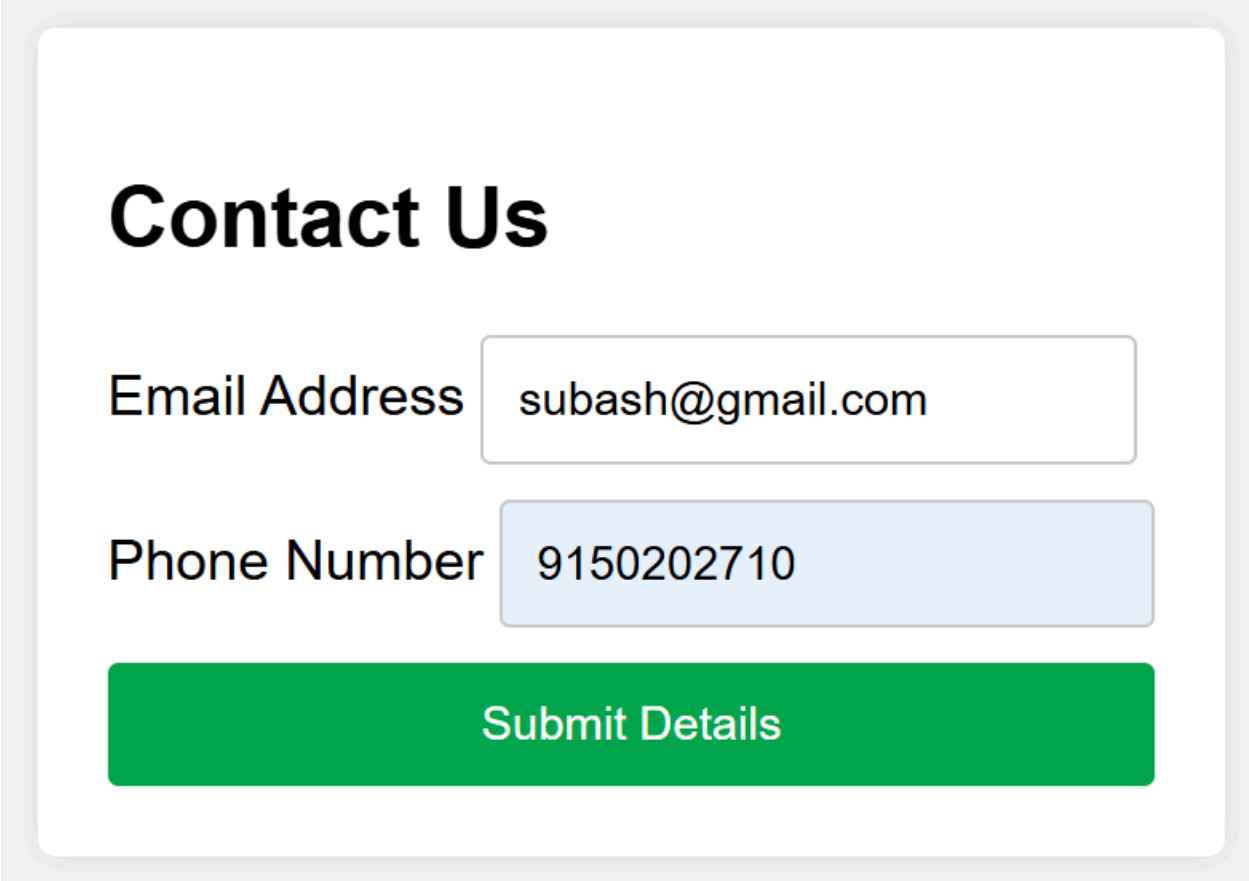
```
    if (!validator.isEmail(email)) {
        emailError.textContent = 'Please enter a valid email
address.';
        isFormValid = false;
    }

    // Execute Mobile Phone Validation
    if (!validator.isMobilePhone(phone, 'any')) {
        phoneError.textContent = 'Please enter a valid phone
number.';
        isFormValid = false;
    }

    // Process data if all validations are successful
    if (isFormValid) {
        console.log('Validation Success!');
        console.log('Submitted Email:', email);
        console.log('Submitted Phone:', phone);
        alert('Form submitted successfully! Check console for
details.');
    }
});
```

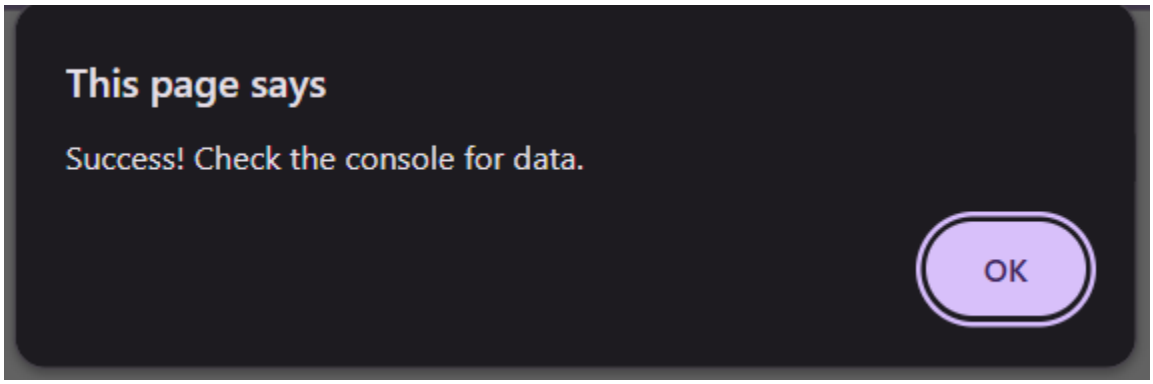

Contact Us
Email Address
e.g., hello@example.com
Phone Number
e.g., +1234567890
Submit Details

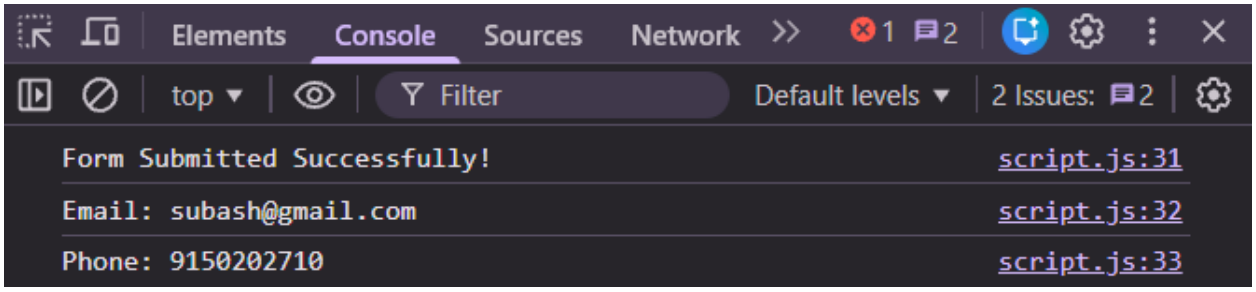

Contact Us
Email Address
subash
Please include an '@' in the email address. 'subash' is missing an '@'.
Submit Details


Contact Us
Email Address
subash@
Please enter a part following '@'. 'subash@' is incomplete.
Submit Details


Contact Us
Email Address
subash@gmail.com
Phone Number
fefef
Please enter a valid phone number.
Submit Details


Contact Us
Email Address
subash@gmail.com
Phone Number
9150202710
Submit Details


This page says
Success! Check the console for data.
OK


Elements
Console
Sources
Network
1
2
top
Filter
Default levels
2 Issues: 2
Form Submitted Successfully!
script.js:31
Email: subash@gmail.com
script.js:32
Phone: 9150202710
script.js:33